

Frontend Website Specification

Architecture, Page Layouts, Navigation Workflows, Core Features, Data Requirements, and Future Enhancements

Aviation Safety Atlas / Atlas Argus

Prepared: 2026-05-29

Core product principle

The projected public record is not the truth. The evidence chain is the truth. The projection is the current best explanation of that evidence.

Document purpose: define the frontend product architecture, public website, authenticated app workspace, BFF boundary, navigation model, page-by-page requirements, data contracts, workflows, testing expectations, and future enhancement roadmap.

1. Product Scope

The frontend should be delivered as two separate surfaces: a public marketing website for trust-building and qualified lead capture, and an authenticated web application for investigative review, claims, conflicts, provenance, timelines, reports, document ingestion, and audit views.

The current frontend is not a blank slate. The uploaded app scaffold already includes React, Vite, TypeScript, Tailwind, TanStack Query, React Router, and routes for login, cases, claims, conflicts, timeline, provenance, reports, and audit. Preserve and harden this scaffold rather than rebuilding it from scratch.

Surface	Primary users	Purpose	Deployment shape
Public marketing website	Prospects, buyers, security/legal reviewers	Explain the product, establish credibility, and capture qualified requests.	Separate static app under apps/marketing
Authenticated web app	Analysts, reviewers, admins	Investigate cases, inspect evidence, resolve conflicts, generate reports, and audit decisions.	Existing apps/web React/Vite SPA
Backend-for-Frontend	Browser clients indirectly	Own sessions, protect API keys, normalize APIs, aggregate data, and enforce browser-safe workflows.	Separate service under services/bff

2. Frontend Architecture

2.1 Recommended Monorepo Shape

```
apps/  
  web/           # Authenticated React/Vite SPA  
  marketing/     # Public marketing site  
  
services/  
  bff/          # Browser-safe Backend-for-Frontend  
  
packages/  
  ui/           # Shared design primitives, optional later  
  types/        # Shared TypeScript DTOs, optional later
```

2.2 apps/web - Authenticated SPA

Use the existing React/Vite/TypeScript/Tailwind application. The app should remain a pure browser client that talks only to the BFF origin. It should never call FastAPI directly and should never send raw Atlas API keys.

Layer	Technology / pattern	Notes
UI framework	React + Vite + TypeScript	Retain existing scaffold and route structure.
Styling	Tailwind CSS	Use restrained enterprise styling, status chips, dense tables, and resizable panes.
Server state	TanStack Query	Use for caching, invalidation, optimistic conflict updates, and retries.
Routing	React Router	Keep routes thin. Domain logic lives in feature folders.
API client	Central BFF fetch wrapper	credentials: include; camelCase DTOs; stable error shapes.

2.3 apps/marketing - Public Website

The marketing website should be separate from the logged-in SPA. It requires SEO, static delivery, legal/compliance pages, and no dependency on investigative workspace state.

Page	Route	Purpose
Home	/	Explain the product in 10 seconds.

Platform	/platform	Explain claims, conflicts, provenance, timelines, and reports.
Use Cases	/use-cases	Speak to aviation litigation, safety investigators, and airline/MRO safety teams.
How It Works	/how-it-works	Show ingest -> reconcile -> resolve -> export.
Security	/security	Explain tenant isolation, audit hashes, key handling, and deployment posture.
Compliance	/compliance	Describe data handling, retention, sub-processors, and residency.
About	/about	Establish credibility and product rationale.
Request Access	/request-access	Capture qualified leads.
Contact	/contact	General, legal, and security contact.
Legal	/legal/*	Privacy, Terms, AUP, DPA, and sub-processors.

2.4 services/bff - Browser-Safe Security Boundary

Security requirement

The browser-based SPA must never hold or transmit raw X-API-Key payloads. The BFF owns human login, sessions, CSRF protection, and server-side API-key bridging.

Minimum BFF endpoint surface:

```

POST /auth/login
GET /auth/me
POST /auth/logout

GET /api/cases
GET /api/cases/:slug
GET /api/cases/:slug/provenance
GET /api/cases/:slug/conflicts
POST /api/conflicts/:id/resolve
POST /api/conflicts/:id/reopen
GET /api/cases/:slug/timeline
GET /api/cases/:slug/reports
POST /api/cases/:slug/reports/generate
GET /api/reports/:id/download

POST /api/documents
POST /api/documents/:id/ingest
GET /api/documents
GET /api/documents/:id/text

POST /api/leads
POST /api/contact

```

- Browser requests must never include X-API-Key.
- Session cookie must be HttpOnly, Secure, SameSite=Lax, and preferably __Host- prefixed.
- BFF must strip upstream secrets from responses.
- Mutations must require CSRF protection.
- Role and tenant data must come from GET /auth/me.

3. Navigation Model

3.1 Public Website Navigation

Home
Platform
Use Cases
How It Works
Security
Compliance
About
Request Access
Contact
Login

Primary CTA: Request access. Secondary CTA: See how it works. The homepage headline should be: The record you can defend. The public site should emphasize source-traceable evidence and defensibility rather than generic AI claims.

3.2 Authenticated App Navigation

/login
/app
/app/cases
/app/cases/:slug
/app/cases/:slug/claims
/app/cases/:slug/conflicts
/app/cases/:slug/timeline
/app/cases/:slug/provenance
/app/cases/:slug/reports
/app/sources
/app/documents
/app/audit
/app/settings

3.3 Sidebar Structure and Role Visibility

Cases
All Cases
Recent Activity
Workspace
Dashboard
Claims
Conflicts
Timeline
Provenance
Reports
Evidence Intake
Sources
Documents

Ingestion Runs

System

Audit

Settings

Role	UI permissions
Analyst	Read cases, claims, provenance, timeline, reports, and audit. Write actions disabled.
Reviewer	Analyst capabilities plus resolve/reopen conflicts, upload/ingest documents, and generate reports.
Admin	Reviewer capabilities plus destructive/admin workflows, user/key/team configuration, and system operations.

4. Core Page Layouts

4.1 Public Home Page

1. Hero: headline "The record you can defend." and CTAs for Request access and See how it works.
2. Problem: investigation evidence disagrees across NTSB reports, FAA records, maintenance logs, witness statements, FOQA/ASAP, news, and depositions.
3. Solution: claims, conflicts, provenance, timelines, audit trail, and export.
4. Evidence-chain visual: Source -> extracted claim -> conflict -> resolution -> projection -> report.
5. Use cases: aviation litigation, safety investigation, and airline/MRO safety teams.
6. Security strip: tenant isolation, audit hashes, API-key safety, and deployment posture.
7. Request access CTA: short, qualifying lead capture form.

4.2 Authenticated App Shell

```
+-----+
| TopBar: case search, current user, role, environment/status |
+-----+-----+
| Sidebar | Route content |
|         |               |
| Cases   |               |
| Workspace |             |
| Evidence |             |
| System  |             |
+-----+-----+
```

- TopBar includes case search, current user/role indicator, and global loading/error state.
- Do not show fake metrics, decorative AI motifs, or ungrounded confidence claims.
- The route content area is owned by feature modules, not global layout code.

4.3 Case Workspace Layout

```
+-----+-----+-----+
| Left rail | Center focus | Right evidence panel |
|           |           |           |
| Case items | Selected claim/conflict/ | Source, provenance, |
| Claims list | timeline/report preview | claim history, hash |
| Conflicts  |           |           |
| Timeline rows |       |           |
+-----+-----+-----+
```

This three-column layout is the spine of the app. Claims, conflicts, timeline, provenance, and reports should use the same workspace structure so users learn one interaction model.

5. Authenticated Page Specifications

5.1 Login Page

Route: /login. Fields: email, password, submit, error state, and optional request-access link.

```
interface LoginRequest {
  email: string;
  password: string;
}

interface CurrentUser {
  email: string;
  role: "analyst" | "reviewer" | "admin";
  tenantId?: string;
  tenantRole?: "OWNER" | "MEMBER" | "READ_ONLY";
}
```

- 8. User submits credentials.
- 9. BFF validates user.
- 10. BFF sets HttpOnly session cookie.
- 11. App calls /auth/me.
- 12. User lands at /app/cases.

5.2 Case List

Route: /app/cases. Purpose: browse public or tenant-visible accident/event records. Use cursor-based pagination with stable sorting by updatedAt DESC and eventId ASC.

Column	Description
Title	Public display title.
Date	Occurrence date.
Location	Country, region, or location label.
Aircraft	Aircraft type or registration if available.
Operator	Operator if available.
Status	Published, draft, retracted, disputed, or reviewed.
Conflicts	Open conflict count.
Updated	Last projection/publication update.

```
interface CaseListItem {
  slug: string;
  eventId: string;
  title: string;
  occurrenceDate?: string;
  locationLabel?: string;
  aircraftLabel?: string;
  operatorLabel?: string;
  fatalityCount?: number;
  injuryCount?: number;
  publicationStatus: "published" | "draft" | "retracted";
  openConflictCount: number;
  updatedAt: string;
}
```

5.3 Case Dashboard

Route: /app/cases/:slug. Purpose: orient the reviewer before detailed work. Sections: case header, projected summary, key facts, open conflict summary, evidence completeness, recent activity, report status, and document ingestion status.

```
interface CaseDashboard {  
  case: CaseDetail;  
  projectionVersion: number;  
  openConflictCount: number;  
  sourceCount: number;  
  claimCount: number;  
  latestActivity: ActivityItem[];  
  latestReport?: ReportSummary;  
}
```

5.4 Claims View

Route: /app/cases/:slug/claims. Purpose: show the reconciled projected record field-by-field.

- Every displayed structured fact must link to its source claim.
- Manual overrides must be visibly labeled.
- Superseded claims must remain inspectable.
- Confidence must be explained as evidence-support confidence, not causal probability.

5.5 Conflict Queue

Route: /app/cases/:slug/conflicts. Purpose: the main paid workflow. The UI must show explicit conflicts, competing claims, evidence context, and a required rationale before resolution.

```
ConflictQueuePage  
  ConflictFilters  
  ConflictTable  
  ConflictDetailPanel  
    ConflictHeader  
    CompetingClaimsList  
    ResolveForm  
    ReopenForm  
    ConflictHistoryPanel  
  EvidencePanel  
interface ConflictClaimCandidate {  
  claimId: string;  
  fieldName: string;  
  fieldValue: unknown;  
  sourceId: string;  
  sourceName: string;  
  sourceReliabilityTier: number;  
  createdAt: string;  
  claimType: string;  
  isWinning: boolean;  
  isSuperseded: boolean;  
}  
  
interface ResolveConflictRequest {  
  expectedVersion: number;
```

```

winningClaimId?: string;
manualOverrideValue?: unknown;
rationale: string;
}

```

- Submit stable candidate.claimId, never an array index.
- Exactly one of winningClaimId or manualOverrideValue is allowed.
- rationale and expectedVersion are required.
- Analyst role cannot submit; reviewer and admin can submit.
- On HTTP 409, replace panel state with the refreshed conflict and force re-review.

5.6 Evidence Panel

The EvidencePanel is reused by claims, conflicts, timeline, provenance, and reports. It must keep the evidence chain visible without breaking the user workflow.

```

interface EvidencePanelData {
  selectedClaimId?: string;
  source: {
    sourceId: string;
    name: string;
    type: string;
    reliabilityTier: number;
    capturedAt?: string;
    ingestionRunId?: string;
  };
  claim: {
    claimId: string;
    fieldName: string;
    rawValue: unknown;
    normalizedValue?: unknown;
    claimType: string;
    lifecycleState: string;
    createdAt: string;
  };
  provenance: ProvenanceStep[];
  claimHistory: ClaimHistoryItem[];
  auditHash?: string;
  linkedConflicts?: ConflictSummary[];
}

```

5.7 Timeline View

Route: /app/cases/:slug/timeline. Purpose: reconstruct a sequence from evidence-backed timeline entries.

- Chronological list with source-backed entries.
- Conflict indicators for uncertain or disputed events.
- Filters by source, certainty, time range, and event type.
- Clicking an entry updates the EvidencePanel.

```

interface TimelineItem {
  id: string;
  occurredAt?: string;
  label: string;
  description?: string;
}

```

```

sourceIds: string[];
claimIds: string[];
hasConflict: boolean;
confidenceLabel?: string;
}

```

5.8 Provenance View

Route: /app/cases/:slug/provenance. Purpose: inspect the evidence chain directly through a tree view, claim history table, source verification panel, and audit hash copy action.

```

interface ProvenanceTree {
  eventId: string;
  projectionVersion: number;
  fields: ProvenanceField[];
}

interface ProvenanceField {
  fieldName: string;
  projectedValue: unknown;
  winningClaimId?: string;
  competingClaimIds: string[];
  conflictId?: string;
  history: ClaimHistoryItem[];
}

```

5.9 Reports Page

Route: /app/cases/:slug/reports. Purpose: generate the sellable deliverable: a defensible, versioned export. Reports must be immutable snapshots of what was believed at a point in time, built from projection, provenance, conflict history, and audit data.

- HTML preview before generation.
- Generate immutable PDF.
- List historical versions.
- Download authorized reports.
- Show projection version and content hash.
- Include disclaimer and source footnotes for every output fact.

```

interface ReportSummary {
  reportId: string;
  eventId: string;
  version: number;
  projectionVersion: number;
  contentHash: string;
  generatedAt: string;
  generatedBy: string;
  downloadUrl: string;
}

interface ReportPreview {
  html: string;
  projectionVersion: number;
  warnings: string[];
}

```

```
}
```

5.10 Documents / Sources

Routes: /app/documents and /app/sources. Purpose: get real evidence into the system. The UI must support upload, parse status, extracted text review, parse failures, ingest action, source linkage, and display of resulting claims/conflicts.

```
interface UploadedDocument {  
  documentId: string;  
  eventId?: string;  
  filename: string;  
  mimeType: string;  
  sizeBytes: number;  
  status: "uploaded" | "parsed" | "parse_failed" | "ingested" | "ingest_failed";  
  extractedTextAvailable: boolean;  
  sourceId?: string;  
  createdAt: string;  
  updatedAt: string;  
}
```

5.11 Audit View

Route: /app/audit. Purpose: make integrity visible to non-technical users. Include activity filtering by case, actor, action, and date; conflict resolution history; report generation history; source ingestion history; and hash verification.

```
interface AuditItem {  
  auditId: string;  
  actorId: string;  
  actorLabel?: string;  
  role: string;  
  action: string;  
  entityType: string;  
  entityId: string;  
  eventId?: string;  
  createdAt: string;  
  auditHash?: string;  
}
```

6. Core Workflows

6.1 Request Access Flow

```
Visitor lands on marketing site  
-> Clicks Request access  
-> Completes qualifying form  
-> BFF stores lead  
-> Confirmation shown  
-> Internal team receives notification later  
interface RequestAccessLead {  
  name: string;  
  email: string;  
  organization: string;  
  role: string;
```

```

organizationType:
  | "law_firm"
  | "aviation_safety"
  | "airline"
  | "mro"
  | "insurer"
  | "other";
useCase: string;
message?: string;
}

```

6.2 Login and Session Flow

User opens /app/cases

```

-> No session
-> Redirect to /login
-> Login submitted
-> BFF validates credentials
-> BFF sets HttpOnly cookie
-> App calls /auth/me
-> App loads /api/cases

```

- No API key in browser network requests.
- No API key in JavaScript bundle.
- Unauthenticated /api/* requests return 401 at BFF before FastAPI is reached.
- Logout invalidates the session server-side.

6.3 Conflict Resolution Flow

Reviewer opens case

```

-> Opens Conflicts
-> Selects OPEN conflict
-> Reviews competing claims
-> Selects winning claim OR manual override
-> Enters rationale
-> Submits with expectedVersion
-> Success updates conflict, projection, history, evidence panel

```

409 flow:

Reviewer submits stale expectedVersion

```

-> BFF/API returns 409 with refreshed conflict
-> UI replaces current panel state
-> Shows "This conflict changed while you were reviewing"
-> Reviewer re-evaluates and resubmits with new version

```

6.4 Document Upload and Ingestion Flow

Reviewer uploads PDF/docket

```

-> File stored
-> Parser extracts plain text conservatively
-> Extracted text shown for review
-> Reviewer triggers ingestion
-> Claims created through existing ingestion path

```

- > Conflicts/projections update
- > New evidence appears in claims/conflicts/provenance

6.5 Report Generation Flow

Reviewer opens Reports

- > Preview HTML generated from current projection/provenance/history
- > Reviewer clicks Generate
- > Immutable PDF version stored
- > Report list updates
- > Download available with authorization

7. UI and Interaction Standards

7.1 Visual Tone

Use	Avoid
High-density tables for investigative data	Decorative gradients or glassmorphism
Status chips instead of vague progress bars	Emoji or playful illustrations
Monospace for hashes and IDs	AI sparkle motifs or hype language
Copy buttons for hashes and source IDs	Fake metrics or invented dashboards
Clear primary actions: Resolve, Generate, Upload	Ungrounded confidence claims
Confirmation dialogs for irreversible actions	Hiding provenance behind vague summaries

7.2 Error Handling

Error	UI behavior
401	Redirect to login.
403	Show permission-denied state.
404	Show not-found state.
409	Show refreshed conflict state inline.
422	Show field-level validation.
429	Show rate-limit message.
500	Show stable fallback with support/action ID.

7.3 Loading States

Skeleton states	Inline pending states
Case list	Resolve conflict
Claim table	Reopen conflict
Conflict detail	Upload document
Evidence panel	Generate report
Report preview	Ingest document
Document text extraction	Download report

8. Data Requirements Summary

8.1 Must-Have DTOs

```

CurrentUser
CaseListItem
CaseDetail
ProjectedField
ClaimDetail
ConflictSummary
ConflictDetailResponse

```

```
ConflictClaimCandidate
ResolveConflictRequest
EvidencePanelData
TimelineItem
ProvenanceTree
UploadedDocument
IngestionStatus
ReportPreview
ReportSummary
AuditItem
RequestAccessLead
```

8.2 BFF Responsibilities

- Translate backend snake_case to frontend camelCase.
- Normalize route naming.
- Attach server-side X-API-Key upstream.
- Never expose raw API keys.
- Convert backend errors into stable frontend error shapes.
- Cache event_id <-> slug mapping.
- Enforce sessions and CSRF.
- Aggregate multiple backend calls into frontend-friendly responses where appropriate.

9. Testing and Acceptance Criteria

9.1 Frontend Unit and Component Tests

- Status chips.
- EvidencePanel rendering.
- ConflictCandidate selection.
- ResolveForm validation.
- Role-gated buttons.
- Report list rendering.
- Document upload states.

9.2 BFF Tests

- Login success/failure.
- Session expiry.
- Logout invalidation.
- CSRF enforcement.
- API-key stripping.
- Upstream error normalization.
- Role propagation.

9.3 Playwright E2E Tests

13. Unauthenticated user redirects to login.
14. Login succeeds and /auth/me populates role.
15. Case list renders.
16. Claim detail updates EvidencePanel.
17. Reviewer resolves a conflict.
18. Analyst sees conflict actions disabled.
19. 409 conflict response refreshes the panel and allows retry.
20. Report preview appears.
21. Report generation creates a version.
22. PDF/docket upload shows success, parse failure, and ingest-in-progress states.

10. Future Enhancements

Enhancement	Purpose
Global search	Search cases, claims, sources, and conflicts.
Rich event search	Filter by aircraft, operator, category, date, region, airport, fatalities, status, and provenance quality.
Map UI	Bounding-box events, clustering, hotspots, and map filters.
Causality graph UI	HFACS/SHELO causal factors with evidence-backed assertions.
Orion entity graph	Aircraft, operator, airport, component, and relationship exploration.
Tenant onboarding UI	Teams, tenant roles, private overlays, and tenant source management.
SSO/SAML/OIDC	Enterprise authentication.
Client portal	External access to selected reports/evidence.
NL search	Natural-language query layer with strict grounding verification.
Webhooks	Export, ingestion, and report notifications.
Advanced report templates	Litigation, expert witness, insurance, and safety-review formats.

11. Recommended Build Sequence

23. Wire existing BFF skeleton in atlas-bff-docs-reports.zip: login/me/logout, session cookie, CSRF, server-side API-key bridge.
24. Route normalization: standardize BFF route prefix and ensure apps/web only talks to BFF.
25. App shell hardening: protected routes, sidebar/topbar, case list, role-aware navigation.
26. Case workspace: three-column layout, case dashboard, claims view.
27. Shared EvidencePanel: use across claims, conflicts, timeline, provenance, and reports.
28. Conflict queue hardening: stable claim candidates, correct claimId selection, resolve/reopen, 409 refresh flow.
29. Timeline and provenance: evidence-linked timeline, provenance tree, claim history.
30. Document upload and ingestion: PDF upload, extracted text review, ingestion status, source linkage.
31. Report preview/export: HTML preview, immutable PDF generation, version history, download.
32. Marketing site and request access: static public pages, lead capture, legal/compliance pages.